Práctica 1-Jerarquía de memoria y comportamiento de la memoria caché

Marcelo Fort Muñoz, Alejandro García García

Arquitectura de Computadores
Grupo 03
{marcelo.fort,alejandro.garcia.garcia0}@rai.usc.es

***Resumen*—En esta práctica estudiamos la jerarquía de memoria y el comportamiento de la memoria caché. Para ello, analizaremos el efecto del principio de localidad sobre los accesos a memoria en las prestaciones de programas utilizando el sistema Finisterrae III para la toma de datos mediante la realización de un código en C que nos permita conocer el coste temporal por acceso de datos. Los resultados obtenidos serán objeto de análisis con el fin de observar e interpretar el efecto del sistema de memorias caché del microprocesador teniendo en cuenta conceptos de localidad espacial y localidad temporal. Hemos desarrollado no sólo la parte obligatoria, sino también las otras dos partes optativas.**

***Palabras clave*—Línea caché, coste temporal de lectura, localidad espacial, *prefetching*, optimización de código.**

I. INTRODUCCIÓN

Este trabajo es un resumen de los estudios realizados en un nodo del sistema Finisterrae III [1] para la toma de datos, utilizando el sistema operativo Rocky Linux [2]. El procesador de este sistema es el Intel Xeon® Platinum 8352Y, cuyas características se analizarán en la sección II (Procesador empleado).

El código presentado es un programa en lenguaje C que realiza una operación de reducción de un vector de números aleatorios y mide su tiempo de ejecución. Se utiliza la memoria caché para mejorar el rendimiento del programa. Para ello, se calcula el número de iteraciones necesarias para leer los datos del vector de forma eficiente en la caché.

Los programas que empleamos para la elaboración de la práctica sirven para ilustrar el principio de localidad (se refiere a la idea de que un programa accede con mayor frecuencia a una pequeña porción de su espacio de memoria en un momento determinado).

El principio de localidad temporal (se refiere a la idea de que si se accede a un recurso en un momento determinado, es probable que se acceda de nuevo a ese mismo recurso en el futuro cercano) podemos encontrarlo en los bucles *for* utilizados en el programa, en los que las mismas instrucciones se repiten un número dado de iteraciones; así como en los accesos secuenciales a los elementos del vector A, pues los elementos de este a los que se haya accedido recientemente se almacenarán en caché y estarán disponibles para su acceso inmediato en un futuro cercano.

El programa utiliza argumentos en la línea de comandos para configurar el tamaño del vector y la distancia entre accesos consecutivos en la caché.

La información sobre las cachés de nuestro procesador también ha sido recogida, como se muestra en la tercera parte de este informe, ya que esta era necesaria para la realización de la práctica.

También se han analizado los códigos, así como su ejecución, con diferentes valores para los parámetros que representan R, L y D en el apartado IV del programa. En estos programas se han tratado distintos tipos de datos (*int* y *double*), además de utilizar diversas formas de acceso a memoria para llegar a estos (acceso mediante índice y acceso directo).

Los resultados de los citados programas han sido objeto de estudio, observando como varían dependiendo del tipo de dato utilizado o de la forma de acceso a las variables del programa, como aparece expuesto en el quinto apartado de este informe (Resultados).

II. PROCESADOR EMPLEADO

El procesador empleado para la ejecución de los programas es **Intel Xeon® Platinum 8352Y**, que cuenta con 32 núcleos por socket (zócalo), 2 *threads* por núcleo (lo cual suma un total de 64 *threads* en el procesador), una frecuencia máxima por núcleo de 3.40 GHz y una frecuencia básica de 2.20 GHz y un total de 48 MiB de memoria caché en tres niveles (la cual se analizará en mayor profundidad posteriormente). También hay que destacar que cuenta con 8 canales de memoria (caminos de acceso que tiene el procesador a la memoria RAM del sistema), lo que hace que exista una alta velocidad de transferencia de datos entre la memoria y la CPU.

En lo que a la caché respecta, se organiza en tres niveles: L1, que, a su vez, se divide en la caché L1 de instrucciones y la caché L1 de datos; L2 y L3.

III. OBTENCIÓN DE LA INFORMACIÓN SOBRE LAS CACHÉS DEL PROCESADOR

Para conocer los valores de los distintos niveles caché utilizamos los datos del comando *getconf -a* y la información obtenida del sitio web *cpu-world* [3] que nos permitió obtener los siguientes resultados sobre la memoria caché:

Caché L1d (L1 de datos):	**1 MiB ***
Caché L1i (L1 de instrucciones):	**1.5 MiB ****
Caché L2:	**40 MiB *****
Caché L3:	**48 MiB ******
Tamaño de línea L1:	**64 Bytes**
Tamaño de línea L2:	**64 Bytes**

*** 32 KiB para cada uno de los 32 núcleos (*cores*).**
**** 48 KiB para cada uno de los 32 núcleos (*cores*).**
***** 1,25 MiB por núcleo, común para datos e instrucciones.**
****** Compartida entre todos los núcleos.**

Para realizar el experimento se ha recalcado la importancia del cálculo del número de líneas de caché de las que consta el procesador. Usando el comando `getconf -a` obtuvimos los siguientes parámetros:

LEVEL1_DCACHE_SIZE	49152 (Bytes)
LEVEL1_DCACHE_ASSOC	12
LEVEL1_DCACHE_LINESIZE	64 (Bytes)
LEVEL2_CACHE_SIZE	1310720 (Bytes)
LEVEL2_CACHE_ASSOC	20
LEVEL2_CACHE_LINESIZE	64 (Bytes)

En base a estos datos, estos han sido nuestros cálculos:

```
S1(N° de líneas Nivel 1): 49152/64 = 768

S2(N° de líneas Nivel 2): 1310720/64 = 20480
```

Tengamos en cuenta que la caché L3 es de 48 MiB.
Aunque es compartida para todo el procesador, somos los únicos que estamos usándolo en ese momento, se supone que buena parte de esa podría usarse en nuestro programa.
La caché L3 tiene 786 432 líneas, lo que equilvaldría a llenar 38,40 veces la caché L2.

Un punto fundamental también a estudiar es, que las caches del procesador son asociativas por conjuntos, lo cual llevará a que en ciertos casos no se puedan aprovechar las cachés al completo como se podrá apreciar en los resultados.

IV. SELECCIÓN DE PARÁMETROS PARA LA EJECUCIÓN DE LOS PROGRAMAS

En este experimento había varios parámetros para tener en cuenta:

L (Número de líneas caché a las que hay que acceder):
Se han especificado los siguientes valores para L:
L= {0.5*S1, 1.5*S1, 0.5*S2, 0.75*S2, 2*S2, 4*S2, 8*S2}.
Siendo:

S1: número de líneas que caben en la caché L1 (768).
S2: número de líneas que caben en la caché L2 (20480).

D (Desplazamiento):
Separación entre datos por lectura. Con D=1 leeremos todos los datos de forma continuada, pero con D=2 significaría que se leerían los datos de forma alterna (Es decir, al sumar 2 al contador uno es saltado cada ronda).

Con D=8, para los *double*, teniendo en cuenta que el tamaño de cada línea de caché es de 64B y que un *double* ocupa 8B, estaríamos leyendo un dato por línea caché, pero para D menor que 8, en ocasiones leeremos más de un dato por línea de caché. Esto será importante a la hora de saber cuántos datos leer para utilizar exactamente cierto número de líneas de caché.

Los valores para este parámetro los hemos seleccionado probando varias potencias de 2 (de 2^0 a 2^{13}). Tras analizar las gráficas para estos valores de D, nos quedamos con aquellos que consideramos más significativos para la visualización del efecto de localidad, dependiendo de si aumentaba o disminuía el desplazamiento.

Los valores de D finalmente seleccionados han sido: **1, 8, 16, 64 y 256**.

R (Nº de datos a los que se ha de acceder para utilizar un determinado número de líneas de caché):

Para el caso de los *doubles*, cuando D es menor que 8, estaríamos leyendo más de un dato en cada línea de caché. Por ejemplo, para D = 1, estaríamos leyendo 8 doubles en cada línea, y para D=2 estaríamos leyendo 4 doubles en cada línea.
Para estos casos, el valor de R ha sido calculado con la siguiente fórmula:

```
ceil((sizeof(double) * (L - 1)) / D) + 1;
```

Cuando esto no sucede, es decir, se lee solamente un dato por línea, podemos hacer que R coincida con el número de líneas caché a las que hay que acceder (L).

V. EJECUCIÓN DEL PROGRAMA

Para ejecutar el programa en el FINISTERRAE III, hemos utilizado el script subido en el Campus Virtual de la asignatura en el que se solicitaban 64 núcleos y 256 GB de memoria.

Hemos desarrollado no sólo la parte obligatoria, sino también las otras dos partes optativas.

Se han ejecutado estas tres versiones del programa en el FINISTERRAE III:

- Una primera con datos tipo *double*
- Otra con datos tipo *int*
- Otra con datos *double* y con acceso sin índice (de forma directa)

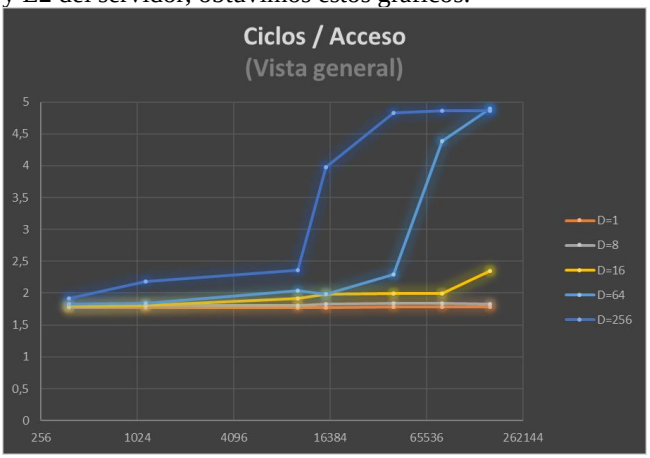
Se ha ejecutado el código con los diferentes valores de D y L que se han seleccionado previamente.

Una vez obtenidos los resultados, hemos creado unos libros de cálculo en Excel y creado gráficas para su visualización.

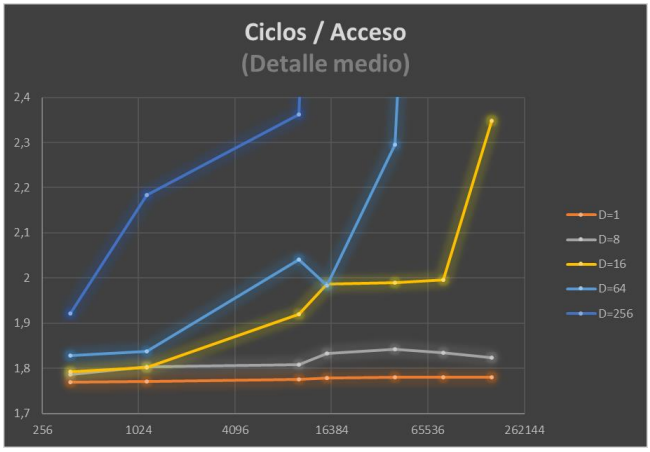
En las gráficas nos hemos asegurado de ajustar los ejes x e y para evitar confusiones gráficas.
Los resultados obtenidos serán expuestos en el siguiente apartado.

VI. RESULTADOS

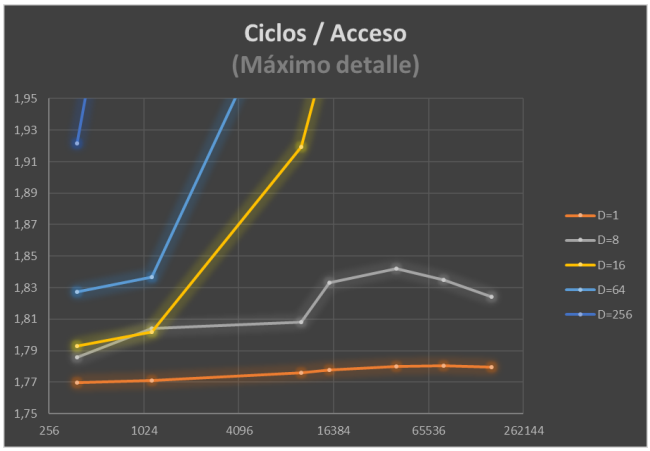
En la primera ejecución, hicimos las pruebas con datos de tipo double y accediendo a las posiciones mediante una variable índice. Tras ejecutarlo para nuestras D y las L que calculamos en el script en base al tamaño de las cachés L1d y L2 del servidor, obtuvimos estos gráficos:



Gráfica 1 Gráfica general con los resultados obtenidos. Escala logarítmica para el eje de las x.



Gráfica 2 Detalle de la gráfica original con el eje y entre 1,7 y 2,4



Gráfica 3 Detalle de la gráfica original con el eje y entre 1,7 y 2,0

Vamos a comenzar por estudiar el resultado para cada uno de los valores de D de forma independiente.

D=1:
En este caso incluso en la gráfica tres se puede ver que el rendimiento es muy lineal. Esto quiere decir que la caché de nivel 1, aunque ha tenido que ir reemplazando su contenido ha hecho un buen trabajo ya que gracias al principio de localidad (hemos utilizado los ocho datos de cada línea) la sistema le ha ido dando tiempo de reponer la información a un ritmo "suficiente" sin importar que tuviéramos o no los datos en caché inicialmente.

D=8:
Vemos que el rendimiento del sistema sigue siendo muy parecido al de D1, aunque en esta ocasión, solo utilizamos un dato de cada línea. Solo se llega a apreciar un aumento del tiempo de acceso del 2%, lo que es prácticamente des-

preciable. Dicho aumento se produce a partir de valores de L mayores que 0.5*S2. **Analizando por qué podría ocurrir esto a partir de ese punto, caemos en la cuenta de que realmente para cada acceso se usan dos líneas de caché: una para el array S y otra para el array A.**

D=16:
Para valores de L pequeños (0.5*S1 y 0.75*S1), donde entra en juego principalmente la cache de nivel 1, los resultados son muy parecidos a los otros valores de D ya analizados previamente. Cuando nos acercamos a los valores en los que se llena S2 (0.5*S2 y 0.75*S2), el número de ciclos aumenta (sobre un 10% para 0.75*S2, manteniéndose en ese nivel hasta 4*S2). Es para el último nivel, con 8*S2 líneas a acceder, donde se disparará hasta 2.35 Ciclos/Segundo.
Para saber por qué ha subido la tasa de ciclos por acceso con L=8*S2, hemos calculado en cuántos conjuntos se divide la caché de nivel 2. Al ser una caché asociativa de 20 líneas por conjunto, las 20 480 líneas de caché se agrupan en 1024 conjuntos.
En el punto en el que L= 8*S2, accederemos a 2*8*S2 líneas.
La razón por la que ha empeorado en este último caso el valor de Ciclos/Acceso, es por que ha tenido que empezar a utilizarse de forma más "fuerte" el reemplazo dentro de cada conjunto de la caché de L2.

D=64 y D=256:
En el caso de D=64, para los niveles de L menores o iguales a 0.75*S2, los resultados son muy parecidos a los obtenidos con D=16.
En el resto de casos, los ciclos necesarios se disparan, llegando a necesitar casi 5 ciclos en el número de accesos.

La notable subida que se puede apreciar en las líneas de la gráfica 2 para los valores de D=64 y D=256 se debe a que estamos cada vez usando menos conjuntos de caché, por estar "saltándonos" ciertas líneas, y que aquellas en las que accedemos están "casualmente" en los mismos conjuntos.

VII. Ejercicio Opcional I

Más tarde, ejecutamos otra versión en la que, en lugar de *double*, los elementos eran de tipo *int*. De esta ejecución surgió la siguiente gráfica:

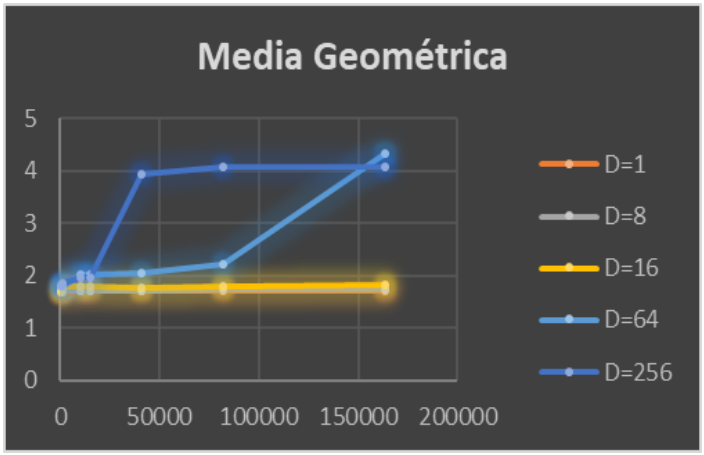
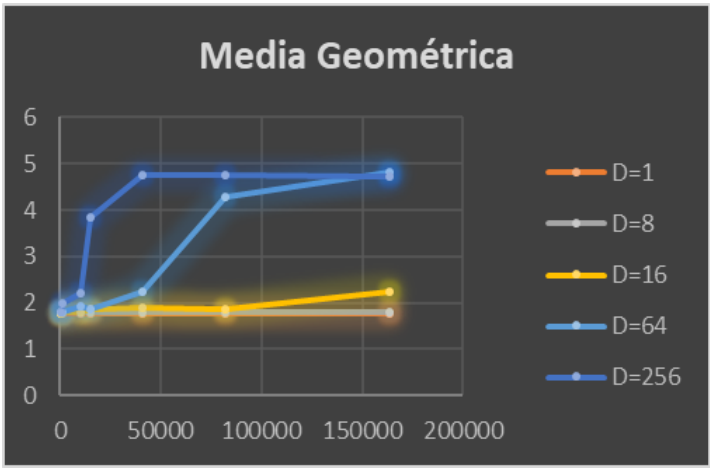


Table 3 Análisis de los datos obtenidos en ejecución con ints e índice

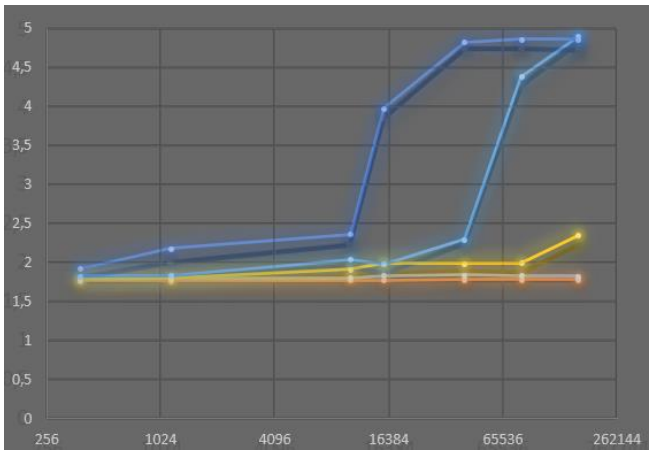
En esta gráfica, se puede ver que se ha reducido la tasa de ciclos/acceso a memoria. Esto se debe a que un entero mide



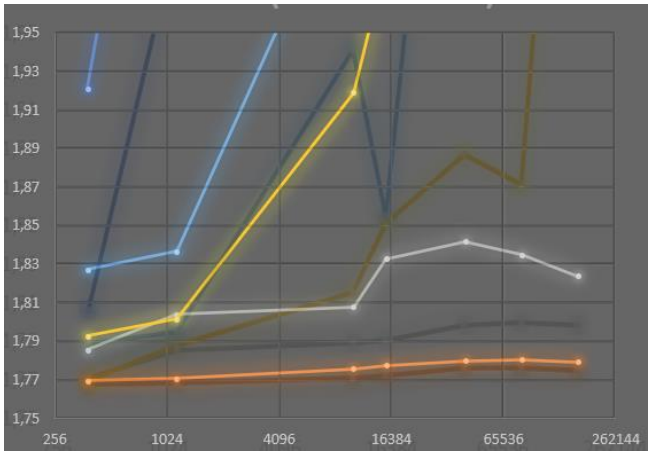
cuatro bytes mientras que un *double* mide ocho bytes. Esto causa que los elementos a acceder se encuentren más cerca y, por lo tanto, el principio de localidad se cumpla en mayor medida. Dicho de otra forma, al ser más pequeños los datos, es más probable que esten en la mísma línea y, como consecuencia, sean más baratos de acceder.

VIII. Ejercicio Opcional II

Después, probamos a realizar la ejecución haciendo un acceso directo a los elementos en vez de referenciar las posiciones con un indice. La gráfica resultante fue la siguiente:



Se puede observar que el hecho de usar un indice (en array) mientras suma dificultad, no es una perdida mayor. Sin embargo, la perdida no es mayor.



IX. CONCLUSIONES

Hemos visto que aprovechar correctamente el principio de localidad espacial y temporal es fundamental a la hora de obtener un código con un rendimiento óptimo.

También hemos visto que cuando hacemos uso consecutivo y reiterado de los datos de una misma línea caché (como nos ha sucedido con D=1) vamos a obtener los mejores rendimientos posibles.

Además, hemos descubierto la importancia de no utilizar potencias "grandes" de 2 como valores de desplazamientos a la hora de acceder a datos, puesto que esto nos lleva a estar utilizando de forma repetida los mismos conjuntos de cache pero para líneas diferentes, forzando el reemplazo de las antiguas (Lo cual nos lleva a en la práctica parecer que tuviéramos cachés mucho más pequeñas).

En cuanto a los ejs. opcionales, hemos visto que:

- Trabajar con datos más pequeños incrementa el principio de localidad.
- Aunque se hagan menos accesos a memoria al no trabajar con índice en array, podemos llegar a tener peores resultados de tiempos de acceso a memoria, pero que lo realmente importante es el tiempo total de ejecución del programa, el cual es más bajo cuando nuestro índice no se encuentra en un array.

REFERENCIAS

[1] CESGA, «Cesga,» [En línea]. Available: https://www.cesga.es/infraestructuras/computacion/.

[2] Rocky Enterprise Software Foundation, «Rocky Linux,» [En línea]. Available: https://rockylinux.org/. [Último acceso: 16 Marzo 2023].

[3] Cpu world, «Cpu world: Intel Xeon 8352Y specifications,» [En línea]. [Último acceso: Marzo 2023].